

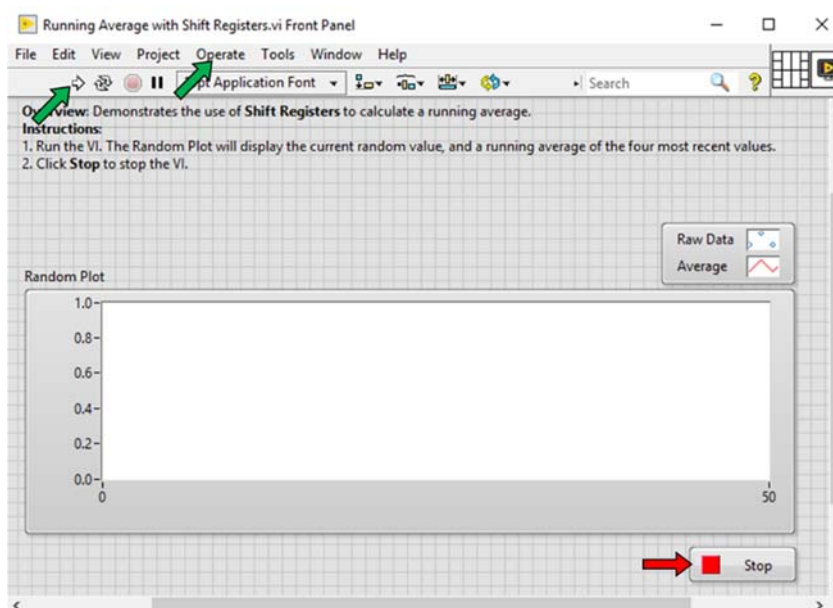
Lab experience 1: Introduction to LabVIEW

LabVIEW is software for the real-time acquisition, processing and visualization of measured data. It can also produce and output signals, such as the voltage that controls each LEDs in a pulse oximeter. A LabVIEW program is called a Virtual Instrument (VI) because, with the proper electronic hardware, it can simulate an instrument such as an oscilloscope or function generator. Each VI is programmed in two windows: a front panel that provides the user interface, and a block diagram that does the data acquisition and mathematics. The block diagram uses a graphical programming interface that mimics the wiring among the electronic components in an electronic device. You can do the same math functions in LabVIEW and MATLAB, but LabVIEW is good for real-time operations while MATLAB is more efficient for processing signals that have already been acquired. The first part of today's exercise is meant to get you familiar with the LabVIEW interface. The second part starts a VI that you can use to do frequency-domain analysis of acoustic signals.

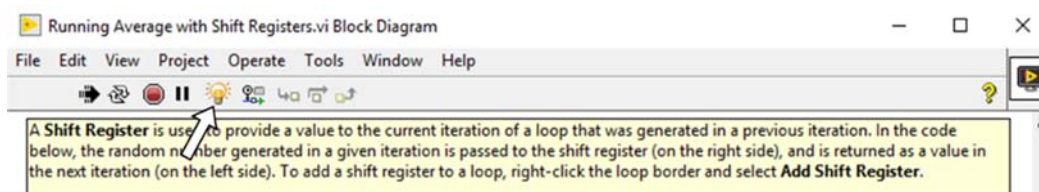
Part 1: Familiarization (Pre-Lab)

Paragraphs in red provide information that is useful but not part of the procedure.

1. Open the application NI LabVIEW 2024.
2. Click Help >> Find Examples in the start window. Click the Search tab, search for "running" then open the Running Average with Shift Registers VI. The first window you see is the Front Panel, which contains one Control (the stop button) and one Indicator (a plot window).
3. Run the VI by clicking Operate >> Run, or ctrl-R, or the white arrow in the upper left corner. The plot shows some random numbers (blue diamonds) and a running average over the most recent four points (orange line). Press the STOP button to stop the VI. Note that the STOP button is different from, and better than, the Abort button, which is the stop sign in the tool bar.



4. If your VI is ready to run, there will be a white arrow in the upper left corner of the window. If not, there will be a broken gray arrow. Clicking on a broken gray arrow will bring up a list of errors so you can address them.
5. Select Window>>Show Block Diagram, or ctrl+E. In this window, the large gray rectangle labeled “Plot Random Number” represents a while loop. The green **T F** box and tiny stop sign determine when the loop stops, and the wristwatch in the yellow box controls how fast it runs. Icons represent functions, numbers, and information passing to or from the front panel. The icons are connected by wires, as if the program were built in a circuit. The wires’ color and pattern indicate the data type that they carry. Some examples:
 - blue for integer, orange for floating point, green for logical (true/false);
 - narrow line for scalar, thick line for 1-D array, double line for 2-D array.The wires also carry information from function to function as if the wires were variables. Values are not given variable names as they are in Java or MATLAB.
6. The orange triangles on the left and right borders are Shift Registers, which store numbers from one iteration of the loop to the next. There are three arrows on the left border because the previous three numbers stored for averaging. Small rectangles represent constants, inputs, or outputs. Rectangles with bold outlines typically represent controls (inputs from the front panel), while rectangles with narrow outlines are indicators (outputs to the front panel).
7. Select Help>>Show Context Help. Note that the Context Help window displays details about whichever component is under the mouse pointer.
8. Right-click the block diagram to bring up the Functions palette, pin it to the desktop, then (if necessary) expand it with the down arrows at the bottom of the window. You might want to Customize >> Change Visible Categories and Select all. Then, select Programming >> Structures, and notice the variety of structures that are analogous to functions in standard programming code. For example, to put functions in a For loop, you place icons and wires into the For Loop structure, or draw the loop around existing icons. Information enters on the left, is processed within the loop, and exits on the right through “tunnels.” The structures of most importance to us are the While Loop, For Loop, Case Structure (equivalent to if-then-else code), and MathScript node. If you do not see the MathScript icon in Programming >> Structures, you might find it under Mathematics >> Scripts and formulas >> Script nodes. The MathScript node allows us to include MATLAB code in the LabVIEW VI so that we do not need to program complicated mathematical formulas icon by icon.
9. While viewing the block diagram, run the VI. While it is running, click the light bulb  (Highlight Execution) and observe the flow of data through the diagram. Note that highlighting execution slows down the VI.



10. Stop and close the Running Average VI, then find the Moonlanding VI in the G:\327\LabVIEW\Lab 1 folder. Play for a while, but not too long.



11. Switch to the block diagram and note the collection of large blue icons. Each of these is an interactive function box, called an Express VI. They can perform signal input & output, mathematics, file input and output, signal processing, and timing.
12. Sometimes an Express VI make our lives easier, especially when dealing with data acquisition and output. Sometimes they are just complicated ways to do simple functions. You can double-click on each Express VI to see its internal settings and options.
13. Before moving to our signal input/output example, note the blue dashed lines traversing the Moonlanding block diagram. This is LabVIEW's "Dynamic Data Type" which means that it can represent any of a variety of signal types. It can be good for plotting acquired signals, but I often like to convert a signal to a simple array because it is easier to manipulate individual samples an array.
14. By default, LabVIEW automatically changes the pointer to match each object (e.g. cross, arrow, text cursor, hand/finger). If you would rather select the pointer yourself, go to the menu bar, select View >> Tools Palette, then click off the green LED above the palette. You can then choose any of the tools shown in the palette. Clicking the LED again restores automatic selection. Either way, try moving or resizing the displays using the arrow pointer.

General notes

- a) On the front panel, pressing the space bar toggles the mouse pointer between the finger (change input) and arrow (move/size display). On the block diagram, the space bar cycles the pointer through the wire and arrow. This can save time and frustration, since you don't have to put the cursor on exactly the right pixel.
- b) *Charts* collect incoming data in a buffer, so one datum (sample) per loop is sufficient. *Graphs* discard all previously received data, so one point per loop is not enough; you need to pass an array to a graph.
- c) If you right click on a wire, the menu includes an "Insert..." option. You can select any function that has an input and output, and it will automatically be placed where you clicked and wired to the correct input and output terminals.
- d) Controls and indicators may be interchanged by selecting "Change to" on the right-click menu. Note that the appearance usually changes on both the diagram and the front panel.
- e) By default, new controls and indicators show up on the block diagram as squares like this . You can make them smaller by right clicking and unchecking "show as icon." Control/indicator icons with bold outlines on the block diagram represent controls on the front panel, while rectangles with narrow outlines represent indicators.
- f) Any VI can be used as a subroutine within a larger VI. Typically, these sub-VIs appear as white squares, which can be opened and edited (right click > Open front panel). They perform a variety of procedures and complicated mathematical functions.

When complicated instrument drivers are involved, the fastest way to get the program you need is often to modify one of the example VI's. Open Help>>Examples>>I/O Interfaces>>DAQ Examples; the Analog Input and Analog Output lists contain the most useful starting points for our data acquisition hardware.

Part 2: *Your* data acquisition and output VI (sound card simulation)

The purpose of Part 2 is to develop a VI that will generate low-frequency signals, pass them through a simulated *sound card* (i.e., the computer's speaker and microphone jacks), and recover the signals for display in the time domain. You may also analyze the component frequencies and plot the magnitude spectrum.

Teams: You are expected to work in pairs, with singlets or triplets as necessary.

Materials:

This simulation can be completed on the computers in the N133 & N151 computer labs. If you wish, you may install LabVIEW on your own computer and proceed with the in-person instructions. However, you will need separate line in & line out jacks (microphone and headphone jacks) in order to plug in an audio cable that passes the signal out of and then back into the computer's sound card.

Documentation:

This lab exercise requires you to modify a VI to produce the desired behavior, and to document your work in your lab notebook, but you do not need to produce a formal lab report with purpose, methods, etc. Your VI will be the primary record of your accomplishments, so each time you successfully add a major feature to your VI, save the VI with a unique name so you can go back to a particular version if you make modifications that cause the VI to stop working. Use your lab notebook to record the major discoveries, accomplishments, and decisions you have made during this lab experience. Specifically, for this lab exercise it will be useful to draw sketches of the signals and flowcharts as you go. The flowcharts can be simple, the sort of signal flow diagram that you might have drawn before you learned about LabVIEW. The signals themselves should be illustrated in both time and frequency domains so you can use your notebook to explain to the instructors what you have done, and why.

VI Starter Package

1. Use Windows Explorer to navigate to G://327/LabVIEW/Lab 1 (yes this one) and open 327_online_output_template.vi, which contains the starting point for your simulated signal input/output VI. Save to your desktop or My Documents.

The **front panel** contains a Waveform Graph that displays the signal before it is sent through the simulated sound card. It also contains the following controls:

- Waveform type (square, triangle, sawtooth, or sine wave)
- Sampling rate and number of samples in this finite-length signal
- Signal frequency (the fundamental frequency of the waveform) *To change the frequency range on the dial, you can double-click on the upper or lower limit and type in a new value.*
- Signal amplitude *You can leave this around 1.*

The **block diagram** contains a “flat sequence” with three frames, contained in a While loop. The first frame generates the signal, the second frame simulates the computer’s sound card (which modifies the signal), and the third frame displays the resulting signal. You can use the Context Help window (ctrl-H) to show you the function of each of the icons in the first frame, since it shows each icon’s name when you mouse over the icon. Pay special attention to the function generator (Signal Processing > Waveform Generation > Basic function generator).

2. Transmit a variety of waveforms, including sinusoids, triangle waves and square waves at various frequencies. Sketch, in your notebook, the waveforms you observe. Be sure to include a low-frequency (2-5 Hz) square wave.

Signal processing challenge

Were you able to transmit the low-frequency square wave through the simulated sound card and get a good-looking square wave on the other side? If so, you’re perplexingly awesome – please show the instructors! If not, what might be causing the shape you observe? How can you upgrade your transmitter and receiver to send a low frequency signal such as this using the sound card? Please discuss with the instructors and proceed to upgrade your VI.

The following tips might be useful for working on signal modulation in frame 1:

3. If you want to create a new waveform (e.g. a sine wave), you may copy and paste (or ctrl+drag) the existing function generator icon and its associated control icons to some open space in the first frame. You will need to do some rearranging of controls on the front panel, so you can keep track of which controls apply to the slow square wave and which apply to the new, faster signal.
4. It is easy to multiply two signals together in LabVIEW: just right click on the wire where you want the multiplication to be, select Insert >> All palettes >> Numeric >> Multiply. The output and one input will be wired up; you just need to wire the other input.
5. It is easy to multiply two signals together in LabVIEW **IF** they have the same sampling rate and number of samples. To ensure that they are the same, you can wire the “Sampling info” wire to any function generator icons you add to the diagram.

The following tips might be useful when working on signal demodulation in frame 3:

6. When placing a sine generator in the receiving frame, please use the sub-VI SineForDemod.vi, which can be found by right-clicking on the block diagram, clicking “Select a VI...” near the bottom of the list, and going to the Lab 4 folder on the G: drive. This function provides some additional realism vis-à-vis the actual computer sound card.
7. It is very important to have the same sampling rate and number of samples in each pair of signals that you multiply (just as having the same number of elements in each array is important when using the .* operator in MATLAB). The easy way to achieve this goal today is to wire the original Sampling Info wire to

our SineForDemod.vi icon. However, this method is not available to us when we use the actual audio hardware. If you are interested in seeing how this is done in the in-person version of the VI, please ask an instructor, and we will be glad to explain.

8. There are many ways to implement frequency filters in LabVIEW. For today, the most user-friendly is probably to right-click on the wire where you want the filter to go, and select Express > Signal Analysis > Filter. This express VI allows you to enter the filter settings you want without connecting any wires and icons.

Write-up and demonstration

By now, you have already saved a few versions of your software as a record and for your future use. You do not need to submit these for a grade, but you do need to record and submit your observations and discussion, preferably hand-written in your notebook or tablet.

In your notebook, you should have recorded the waveforms that you observe when transmitting and receiving VIs before and after the upgrade. Compare these signals to what you expected to see, propose reasons that the signal might be altered, and provide supporting evidence. Include sketches in your notebook of the frequency spectrum showing how the transmitting and receiving stages in your VI manipulated the signal in the frequency domain. You do not need to include formal Purpose, Methods, Results, Discussion sections in your notebook.

Demonstrate your VI(s) for the instructor or TA so that you can get checked off for the exercise. Be prepared to explain how the VI works and the principles behind the signal processing, and to use the frequency domain sketches to support your explanation.

You do not need to submit anything for this activity this week. It will be submitted along with the Fusion360 and COMSOL content in week 4.

Advanced exercises (optional)**1. Frequency analysis**

Now it is time for you to apply your knowledge of discrete Fourier transforms to add frequency-measurement functionality to your VI. ON PAPER, write a flow chart that shows how you will take your signal and determine the dominant frequency (the one with highest amplitude) over the duration when you captured the signal. Be as detailed as you can. Show the flow chart to the instructor and you will get help finding the functions you need.

2. Reading a .wav file

You may modify your VI to read a pre-recorded file rather than generating a waveform. Before doing so, save the VI under a new file name. Delete the Basic Function Generator and its associated controls. Replace it with Programming > Graphics and Sound > Sound > Files > Simple Read, then right click the path input and Create > Control. The data output will probably contain two channels, which should be split with Programming > Array > Index Array. Wire the file data to the Index Array input, then its output should be a single channel that you can use in the same way as you used the signal that your Basic Function Generator created.

Part 2: *Your* data acquisition and output VI (in-lab exercise)

The purpose of Part 2 is to generate a VI that will do the following:

- Ⓟ Send signals (e.g., square waves) from your computer via its sound card
- Ⓟ Receive signals from another computer via your computer's sound card
- Ⓟ Plot the signal in the time domain
- Ⓟ Optional: Analyze the component frequencies and plot the magnitude spectrum.

Teams: You may write your programs individually or in teams of two. You can do all of the signal transmission and capture with one computer.

Materials:

One computer with LabVIEW and line in/line out jacks

One audio cable with 3.5 mm plugs at both ends

Optional: function generator, computer microphone, earphones or speaker.

Documentation:

This lab exercise requires you to create two VIs and document your work in your lab notebook, but you do not need to produce a formal lab report with purpose, methods, etc. Your VIs will be the primary record of your accomplishments, so each time you successfully add a major feature to your VI, save the VI with a unique name so you can go back to a particular version if you make modifications that cause the VI to stop working. Also record the major discoveries, accomplishments, and decisions you have made during this lab experience. For this lab exercise it will be useful to draw sketches of the signals and flowcharts as you go. The flowcharts can be simple, the sort of signal flow diagram that you might have drawn before you learned about LabVIEW. The signals themselves should be illustrated in both time and frequency domains so you can use your notebook to explain to the instructors what you have done, and why.

Signal output

3. Open `sound_output_template.vi`, which contains the basic components of an unfinished sound output VI. The file can be opened from the `G:\327` folder or downloaded from Canvas. Save your re-named copy to the Desktop or U drive.
4. In the While loop on the block diagram, you should find a function generator icon: Signal Processing > Waveform Generation > Basic function generator, along with controls for frequency, amplitude, signal type, and sampling info. The output waveform from the function generator is wired to the input of the Sound Output Write icon. You can use the Context Help window (`ctrl-H`) to help you find the icon, since it shows each icon's name when you mouse over the icon.

5. Optional: use speakers or earbuds to verify the output, but be careful! Low amplitudes or volumes are usually adequate, and sine waves can cause hearing loss at high volume. To change the frequency range on the dial, you can double-click on the upper or lower limit and type in a new value.

Signal acquisition

You could start the acquisition (input) exercise with a microphone, but for efficiency it is probably best if you skip the microphone and go directly to connecting your sound card output to its input so you capture the signal that you generate using your Signal Output VI.

6. If you use a microphone, plug it into the computer, right click the Windows speaker volume icon and select Recording devices, and verify that the green bars indicate sound input.
7. Once you have plugged in your microphone or cable to/from the sound card, start or re-start LabVIEW so it recognizes the input device.
8. Create a blank VI, and switch to the block diagram. You will want to acquire a sound signal, so place this icon on the diagram: Functions > Express > Input > Acquire sound. Use one channel and a sampling frequency of 11025 Hz. Click OK.
9. Expand the Acquire sound icon by dragging down the lower border. Right click the Data output triangle, and Create > Graph indicator.
10. Switch to the front panel and run the VI. After one second you should see one second's worth of signal on the graph.
11. We want your VI to run continuously, so add a While loop on the block diagram (Programming > Structures > While). Drag the loop around both icons. Right click the stop sign in the lower right corner and click Create Control. Double click the Stop icon, and it will be highlighted on the front panel. Run the VI, and notice that you can stop it by pressing the Stop button.
12. On the block diagram, on the Acquire sound icon, right-click the Duration input triangle and Create > Control. On the front panel, enter 0.5 in the Duration (s) field. When you run the VI the chart should update itself every 0.5 seconds.
13. You can multiply the sampling rate by the duration to find the number of samples that are supposed to be in each signal. However, the actual signal often has fewer, and the exact number is very important for frequency analysis. Let's see how many samples we get. Place a Programming > Array > Array size icon near the Data graph icon. Wire the blue Data wire to the Array size input (left side of the icon). You should see a new icon that takes in a blue wire and puts out an orange wire, i.e. changes a Dynamic Data Type to a simple Array. Right click the Array Size output and Create > Indicator. The indicator on the front panel will show the number of samples collected in each iteration of the While loop. With

0.5-second duration, one channel and $f_s = 11025$ Hz, I got 4134 samples on my home computer, but 5512 on my school computer (I don't know why).

14. You should be able to run your VI now and see the incoming signal on the front panel. If the signal jumps around, try stopping the VI, right clicking on the x-axis (time axis) and un-checking Loose Fit.

Combined input & output

15. If you have been using a microphone up to this point, remove the microphone and connect your speaker output to the microphone input of your computer, and verify that your signal acquisition VI can capture the signal from your output VI.
16. Working with your own computer's sound input & output sockets, or with another team and their computer, transmit a variety of waveforms, including sinusoids, triangle waves and square waves at various frequencies. Sketch, in your notebook, the waveforms you observe.
17. Question: Can you reliably transmit a low-frequency square wave – on the order of 1-2 Hz or so? If so, super-duper! If not, what might be causing the shape you observe? How can you upgrade your transmitter and receiver to send a low frequency signal such as this using the sound card? Is this upgrade part of today's lab assignment?¹

You are encouraged to talk to each other and to the instructors about every phase of this VI development.

¹ Yes! Please do it!